

コマ 13: sizeof + malloc + 連結リスト

今日のゴール

`sizeof` と `malloc` を使い、構造体をヒープ上に確保して連結リストを動かす。

第 12 回では、スタック上の構造体変数と `.` / `->` を扱った。この回では、構造体を `malloc` で動的に確保し、ポインタでつないだデータ構造を作る。

```
typedef struct Node {  
    int val;  
    struct Node *next;  
} Node;
```

`Node` は、自分と同じ型へのポインタ `next` を持つ。このような構造体を自己参照構造体と呼ぶ。

1 この回で扱う範囲

対象にする機能は次の通り。

種類	例
<code>sizeof(type)</code>	<code>sizeof(int)</code> , <code>sizeof(Node)</code>
<code>sizeof expr</code>	<code>sizeof *p</code>
<code>malloc</code>	<code>malloc(sizeof(Node))</code>
自己参照構造体	<code>struct Node *next</code>
連結リスト	<code>head = head->next</code>

`malloc` 自体は自作しない。`lib.h` の宣言を使い、リンク時に `libc` の `malloc` を呼び出す。

2 AST を確認する: sizeof

まず、`sizeof` がどのような AST になるか確認する。

```
python3 scaffold/parse_viewer.py sessions/13_sizeof_malloc_list/tests/sizeof_test.
```

このプログラムの内容は次の通り。

```
int main() {  
    return sizeof(int) + sizeof(char);  
}
```

実行すると、次のような S 式が表示される。

```
(program  
  (funcdef "main" :type int (params)  
    (block  
      (return  
        (add (sizeof-type int) (sizeof-type char))))))
```

`sizeof(int)` は `(sizeof-type int)` になる。`sizeof(char)` は `(sizeof-type char)` になる。これらは実行時に計算する必要はなく、コンパイル時に型サイズを見て定数を出せばよい。

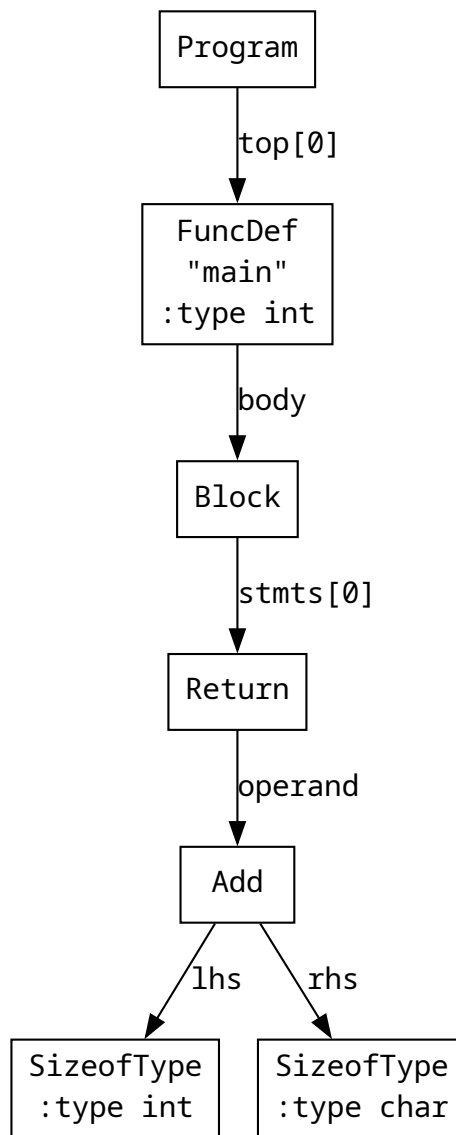
3 AST を確認する: malloc と自己参照構造体

次に、`malloc` と `struct Node *next` を含む小さな例を見る。

```
python3 scaffold/parse_viewer.py sessions/13_sizeof_malloc_list/tests/list_min.c
```

このプログラムの内容は次の通り。

```
int malloc(int size);  
  
typedef struct Node {  
    int val;  
    struct Node *next;  
} Node;
```

図1 `sizeof_test.c` のAST

```
int main() {  
    Node *n;  
    n = malloc(sizeof(Node));  
    n->val = 10;  
    n->next = 0;  
    return n->val;  
}
```

実行すると、次のような S 式が表示される。

```
(program
  (funcproto "malloc" :type int
    (params (param "size" :type int)))
  (funcdef "main" :type int (params)
    (block
      (decl "n" :type
        (ptr (type "Node")))
      (exprstmt
        (assign (var "n")
          (call "malloc"
            (args
              (sizeof-type (type "Node"))))))
      (exprstmt
        (assign
          (member "->" "val" (var "n"))
          (num 10)))
      (exprstmt
        (assign
          (member "->" "next" (var "n"))
          (num 0)))
      (return
        (member "->" "val" (var "n"))))))
```

`malloc(sizeof(Node))` は、`sizeof(Node)` の結果を引数として `malloc` を呼び出すだけである。戻り値はポインタ値なので、`Node *n` に代入できる。

4 sizeof のコード生成

`sizeof` は値を計算する式だが、実行時にメモリを読みに行く必要はない。型情報からサイズを調べ、即値を `a0` に入れる。

```
if node.kind == ND_SIZEOF_TYPE:
    self.emit(f" li a0, {self.size_of_ty_str(node.ty_str, self._struct_defs)}")
    return

if node.kind == ND_SIZEOF_EXPR:
    ty_str = self._type_of_expr(node.operand)
    self.emit(f" li a0, {self.size_of_ty_str(ty_str, self._struct_defs)}")
    return
```

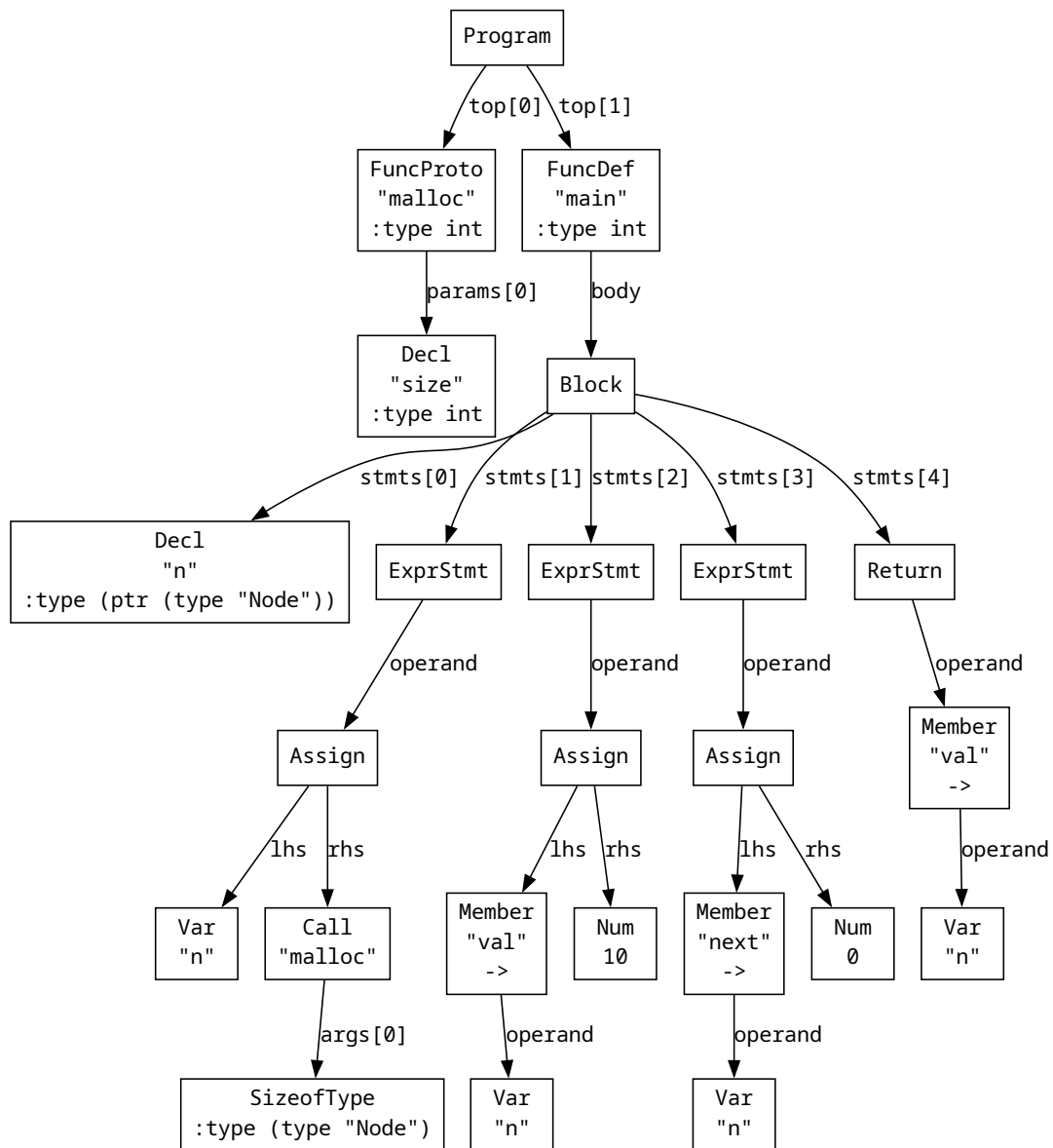


図2 list_min.c のAST

たとえば `sizeof(Node)` は、`self._struct_defs["Node"]["size"]` が 8 なら `li a0, 8` を出す。

5 malloc は普通の関数呼び出し

`malloc` はライブラリ関数である。コンパイラが行うことは、通常の関数呼び出しと同じである。

```
n = malloc(sizeof(Node));
```

この式では、まず `sizeof(Node)` を `a0` に計算し、それを第 1 引数として `malloc` を呼び出す。戻り値も `a0` に返る。

6 malloc を呼び出すと何が起こるか

`malloc` は、指定されたバイト数ぶんのメモリ領域をヒープから確保し、その先頭アドレスを返すライブラリ関数である。

```
n = malloc(sizeof(Node));
```

この例では、まず `sizeof(Node)` によって `Node` 1 個ぶんに必要なバイト数を求める。その値を第 1 引数として `malloc` に渡す。`malloc` は実行時に、そのサイズ以上の連続したメモリ領域をヒープから確保し、その先頭アドレスを返す。

返ってきた値は、構造体そのものではなく、確保されたメモリ領域の先頭アドレスである。そのアドレスを `Node *n` に代入することで、以後その領域を `Node` として扱える。

```
n->val = 10;  
n->next = 0;
```

この代入によって、確保した領域の中に実際のフィールド値を書き込んでいる。

重要なのは、`malloc` はメモリを確保するだけで、中身を初期化しないことである。確保直後の `n->val` や `n->next` の値は未定義なので、使う前に自分で代入する必要がある。

実際の `malloc` の内部では、未使用のヒープ領域を管理し、要求されたサイズに合う領域を探して返している。多くの実装では、確保した領域の前後に管理用の情報を持ったり、足りない場合に OS から追加のメモリを取得したりする。ただし今回のコンパイラでは、その仕組みを実装しない。コンパイラが担当するのは、引数を `a0` に置き、呼び出し規約に従って `libc` の `malloc` を呼び出すところまでである。

なお、本来は `malloc` で確保したメモリは、不要になったら `free` で解放する。この回では、プログラムがすぐ終了する小さな例だけを扱うため、`free` は実装範囲に含めない。

7 自己参照構造体

`Node` の定義には `struct Node *next` が含まれている。

```
typedef struct Node {
    int val;
    struct Node *next;
} Node;
```

`next` は `Node` そのものではなく、`Node` へのポインタである。ポインタのサイズは常に 8 バイトなので、構造体の中に自分自身へのポインタを持てる。

レイアウトは次のようになる。

フィールド	型	オフセット	サイズ
<code>val</code>	<code>int</code>	0	4
<code>next</code>	<code>struct Node *</code>	8	8

`next` は 8 バイト境界に置くため、`val` の後に 4 バイトの余白が入る。この簡易実装では、各フィールドをその型サイズに応じて自然な境界に揃える。

8 struct tag と typedef 名

第 12 回では、主に `typedef struct { ... } Point;` を扱った。第 13 回では `tag` 名付きの構造体を扱う。

```
typedef struct Node { ... } Node;
```

この場合、次の 2 つの名前で同じ構造体型を参照できるようにする。

名前	使われ方
<code>Node</code>	typedef 名
<code>struct Node</code>	struct tag 名

つまり、次の 2 つの呼び出しが同じサイズを返すようにする。

```
# typedef 名でも struct tag 名でも同じサイズが返る
self.size_of_ty_str("Node", self._struct_defs)
self.size_of_ty_str("struct Node", self._struct_defs)
```

9 連結リストの走査

連結リストは、各ノードが次のノードへのポインタを持つ構造である。

```
int list_sum(Node *head) {
    int sum;
    sum = 0;
    while (head != 0) {
        sum = sum + head->val;
        head = head->next;
    }
    return sum;
}
```

`head != 0` は、ポインタが `NULL` でないことを調べている。`head->val` で現在のノードの値を読み、`head = head->next` で次のノードへ進む。

補足

`while (head != 0 && head->val > 0)` とは書けない。このスキファールドの `&&` は両辺を必ず評価するため、`head` が `NULL` のときに `head->val` を読んでクラッシュする。`NULL` の判定と中身の判定は、上の例のように分けて書くこと（短絡評価の実装は発展課題 S1 で扱う）。

`head` はスタック上のローカル変数だが、各ノードは `malloc` で確保したヒープ上にある。`head = head->next` は、`head` の指す先を矢印 1 つぶん右のノードへ進める。

10 実装手順

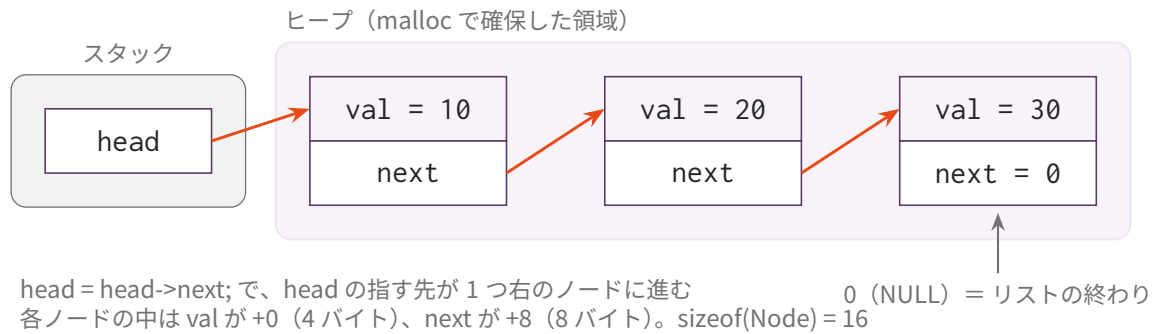


図3 スタック上の head とヒープ上に確保された 3 ノード

1. スケルトンの `importlib` 継承により第 12 回の `Codegen` クラスを引き継ぐ（あらかじめ書かれている）
2. `CodegenNN.register_typedef_names()` / `parse_struct_defs()` を tag 名付き `struct` に対応させる
3. 自己参照用に、フィールド走査前に仮のエントリを `self._struct_defs` に登録する
4. `self.size_of_ty_str()` で `struct Node` と `Node` を同じサイズへ解決する
5. `sizeof(type)` と `sizeof expr` が構造体サイズを返すことを確認する
6. `malloc(sizeof(Node))` が通常関数呼び出しとして動くことを確認する
7. `list_sum.c` まで通す

11 テスト

```
python3 scaffold/test_runner.py sessions/13_sizeof_malloc_list
```

この回の主要テストは次の通り。

テスト	内容	期待値
<code>sizeof_test.c</code>	<code>sizeof(int) + sizeof(char)</code>	5
<code>malloc_struct.c</code>	<code>malloc(sizeof(Box))</code> と <code>-></code>	17
<code>list_sum.c</code>	<code>Node</code> の連結リスト走査	60